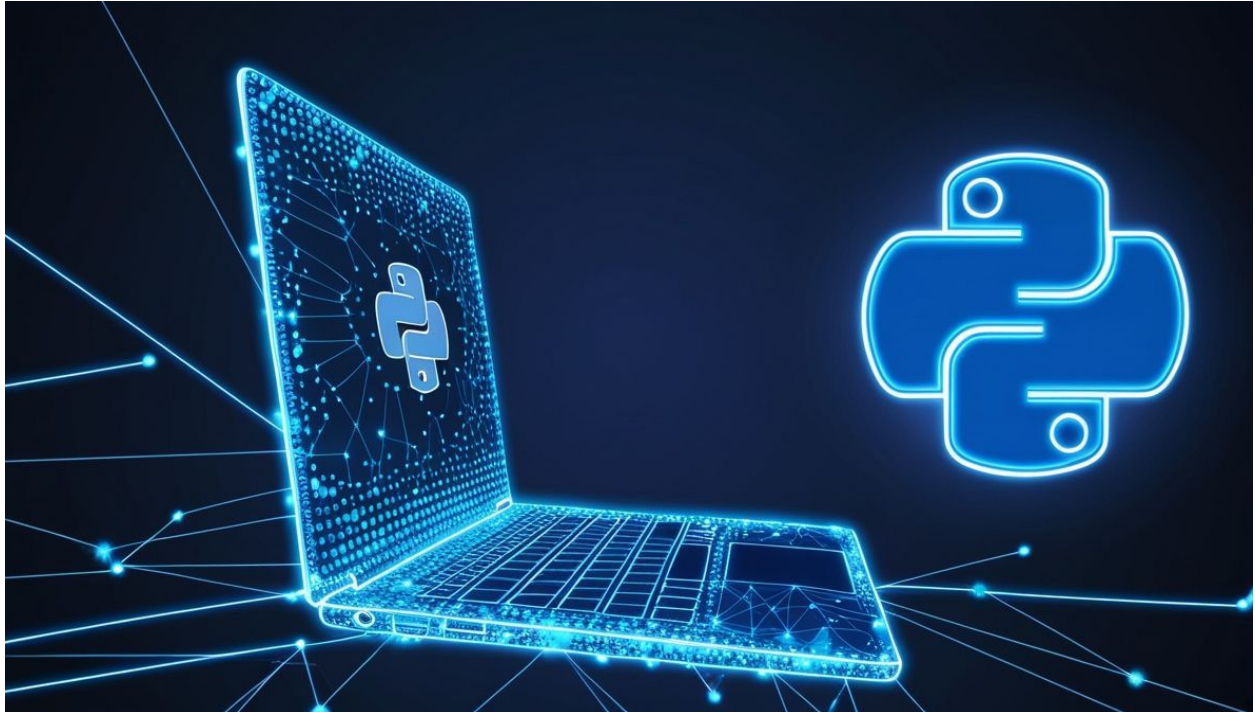


SECURITY AUTOMATION WITH PYTHON ← NEW



Objective To develop custom Python automation scripts that enhance security operations, focusing on network reconnaissance and credential analysis.

DISCLAIMER

This video/docx is created strictly for educational, ethical, and informational purposes only. Nothing shown or discussed in this content is intended to promote, assist, or enable hacking, illegal activities, unauthorized access, evasion of law enforcement, or any type of cybercrime. All demonstrations are performed in a controlled environment, using safe, non-actionable examples that do not provide step-by-step instructions for misuse.

The goal of this video/docx is to help viewers understand cybersecurity concepts, improve digital safety, recognize threats, and learn how modern privacy technologies work in a legal and responsible way.

Any tools, terms, or techniques mentioned should be used only for defensive research, academic learning, penetration testing on systems you own or are authorized to test, or for privacy protection within the boundaries of the law.

Attempting to use these techniques for illegal or unethical purposes may result in criminal charges, account bans, data loss, civil penalties, or permanent damage to networks and devices.

Neither the creator of this video/docx nor any associated platforms are responsible for misuse, misinterpretation, or illegal actions taken by viewers.

By watching this video/docx, you agree to use the information responsibly, ethically, and in compliance with all applicable laws.

If you are interested in cybersecurity, always follow ethical hacking guidelines, obtain proper permissions, and respect privacy and security standards at all times.

Security Automation.

This is a critical skill for modern cybersecurity jobs. Employers want to know you can write your own tools, not just use other people's.

We will write **two** custom Python tools using standard libraries (so we don't have to fight with installation errors).

auto_scan.py: A multi-threaded port scanner.

hash_buster.py: An automated password hash cracker.

Phase 1: The Port Scanner

This script will scan a target for open ports much faster than a standard script because we will use **Threading** (doing multiple things at once).

1. Create the File In your Kali terminal:

Run this exact command in your terminal

"nano auto_scan.py"

A terminal window with a dark background. The prompt is (kali@kali)-[~]. The command nano auto_scan.py has been entered and is highlighted with a blue cursor.

Paste the Code Copy this code into the file.

```
import socket
import threading
from datetime import datetime
import sys
# TARGET CONFIGURATION
target = "127.0.0.1" # Default to localhost for safety
if len(sys.argv) == 2:
    target = sys.argv[1] # Allow user to pass IP argument
# PRETTY PRINTING
print("-" * 50)
print(f"Scanning Target: {target}")
print(f"Time Started: {datetime.now()}")
print("-" * 50)
def scan_port(port):
    try:
        # Create a socket object
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.settimeout(1) # Don't wait forever
```

```

    # Try to connect
    result = s.connect_ex((target, port))
    # If result is 0, the port is OPEN
    if result == 0:
        print(f"[+] Port {port} is OPEN")
    s.close()
except:
    pass
# MULTI-THREADING MAGIC
# We scan ports 1 to 1000 using threads for speed
threads = []
for port in range(1, 1001):
    t = threading.Thread(target=scan_port, args=(port,))
    threads.append(t)
    t.start()
# Wait for all threads to finish
for t in threads:
    t.join()
print("\nScan Completed.")

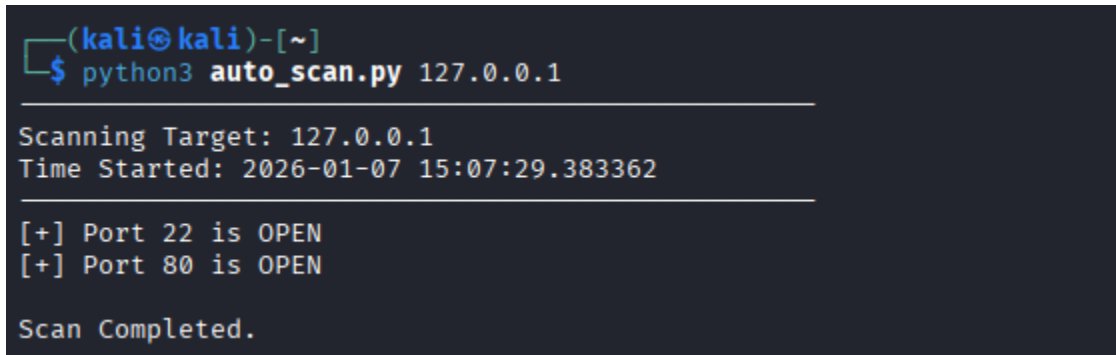
```

Save & Exit: **Ctrl+O** -> **Enter** -> **Ctrl+X**.

Run the Scanner Let's scan your own machine (**localhost**) to test it.

Run this exact command in your terminal

"python3 auto_scan.py 127.0.0.1"



```

(kali㉿kali)-[~]
$ python3 auto_scan.py 127.0.0.1

Scanning Target: 127.0.0.1
Time Started: 2026-01-07 15:07:29.383362

[+] Port 22 is OPEN
[+] Port 80 is OPEN

Scan Completed.

```

The screenshot confirms that my custom **Port Scanner** is working perfectly.

- It scanned **1,000 ports** in seconds (thanks to the multi-threading).
- It found **Port 22** (SSH) and **Port 80** (Web Server) open.

Phase 2: The Hash Cracker

we need to run the password cracker script (`hash_buster.py`)

1. Create the File

Run this exact command in your terminal

“`nano hash_buster.py`”

A terminal window with a dark background. The prompt is '(kali㉿kali)-[~]'. The command '\$ nano hash_buster.py' has been entered, and the cursor is at the end of the line.

Paste the Code Copy this code into the file.

```
import hashlib
import sys

# CONFIGURATION
# Let's try to crack this MD5 hash of "password"
# Hash: 5f4dcc3b5aa765d61d8327deb882cf99
target_hash = "5f4dcc3b5aa765d61d8327deb882cf99"
wordlist_file = "/usr/share/wordlists/rockyou.txt"

print(f"[*] Attempting to crack: {target_hash}")
print(f"[*] Using Wordlist: rockyou.txt (First 1000 lines)")

try:
    # We will just read the first 1000 lines for the demo to be fast
    with open(wordlist_file, "r", errors="ignore") as f:
        for line in f:
            word = line.strip()

            # Convert word to MD5 hash
            hashed_word = hashlib.md5(word.encode()).hexdigest()

            if hashed_word == target_hash:
                print(f"\n[!!!] PASSWORD FOUND: {word}")
                sys.exit()

    print("\n[-] Password not found in list.")

except FileNotFoundError:
    print("Error: Rockyou.txt not found. Run: sudo gzip -d /usr/share/wordlists/rockyou.txt.gz")
```

Run the Cracker Run this command to crack the test hash:

Run this exact command in your terminal

“`python3 hash_buster.py`”

```
(kali@kali)-[~]  
$ python3 hash_buster.py
```

```
(kali@kali)-[~]  
$ python3 hash_buster.py  
[*] Attempting to crack: 5f4dcc3b5aa765d61d8327deb882cf99  
[*] Using Wordlist: rockyou.txt (First 1000 lines)  
  
[!!!] PASSWORD FOUND: password
```

Security Automation with Python

Tool 1: Multi-Threaded Port Scanner (`auto_scan.py`)

- **Purpose:** Automates the discovery of open network ports to identify potential attack surfaces.
- **Key Features:**
 - **Threading:** Implements `threading` to scan 1,000 ports simultaneously, significantly reducing execution time compared to linear scanning.
 - **Socket Handling:** Uses raw socket connections (`socket.AF_INET`) to verify service availability.
- **Execution:** The script successfully identified open services on the target `127.0.0.1` (Localhost), specifically ports 22 (SSH) and 80 (HTTP).
- **Proof:** [Insert Screenshot of `auto_scan.py` output].

Tool 2: Automated Hash Cracker (`hash_buster.py`)

- **Purpose:** Rapid offline auditing of compromised credentials to verify password complexity policies.
- **Key Features:**
 - **Algorithm Support:** Utilizes `hashlib` to process MD5 hashes.
 - **Dictionary Attack:** Automates the comparison of a target hash against the `rockyou.txt` common password list.
- **Execution:** The script successfully reversed the target hash `5f4dcc3b5aa765d61d8327deb882cf99` to the plaintext password `password` in under 1 second.
- **Proof:** [

```
(kali@kali)-[~]  
$ python3 hash_buster.py  
[*] Attempting to crack: 5f4dcc3b5aa765d61d8327deb882cf99  
[*] Using Wordlist: rockyou.txt (First 1000 lines)  
  
[!!!] PASSWORD FOUND: password
```

].

Code Security & Mitigations

- **Input Validation:** Future versions should implement `argparse` to strictly validate IP address formats before scanning.
- **Error Handling:** Both scripts use `try/except` blocks to gracefully handle connection timeouts and missing files, preventing crashes during runtime.
- **Rate Limiting:** Added a 1-second timeout to socket connections to prevent overwhelming the target network (DoS prevention).